

Svolgere gli esercizi e la relativa sperimentazione numerica in Matlab; consegnare un file con estensione .m per ciascun esercizio, contenente, nell'ordine, lo script e la/le function richiesta/e dall'esercizio, inserendo eventuali commenti da esso richiesti in righe commentate in fondo al file. Riportare l'etichetta del gruppo e i nomi e cognomi dei componenti del gruppo.

Esercizio 1

- Scrivere una **function** che, presa in input una matrice A simmetrica (non necessariamente sdp) di dimensione $n \times n$, se possibile, ne produca in output la fattorizzazione LDL^T (con D avente non necessariamente elementi positivi sulla diagonale), fornendo in caso contrario un opportuno messaggio di errore. Fornire in output anche un flag che, se l'algoritmo arriva a termine, venga settato pari a 1 o a 0, rispettivamente, indicando che la matrice A è o non è sdp.
- Realizzare uno **script** che, utilizzando la function del punto precedente, risolva efficientemente il sistema lineare con matrice dei coefficienti data dalla matrice di Hilbert (usare il comando predefinito `hilb`) di dimensione $n \times n$, al variare di $n = 5, 6, \dots, 13$, considerando come soluzione esatta del sistema il vettore con tutte componenti uguali a 2. Riportare in un file di report i valori del flag, dell'errore relativo, del residuo relativo e del numero di condizionamento di A al crescere di n (lavorare in norma euclidea). Commentare i risultati ottenuti sia dal punto di vista del flag (tenere presente che si può dimostrare che le matrici di Hilbert sono sdp) che delle altre quantità stampate.

Esercizio 2

- Scrivere una **function** Matlab che, presi in input un flag di nome **itype** (con valore 0 o 1), un intero positivo n e un set ordinato di punti nel piano $\mathbf{p}_i, i = 1, \dots, m$, con $m > n$, fornisce in output il set di punti nel piano $\mathbf{c}_j, j = 0, \dots, n$, tali che la curva parametrica $\mathbf{X}(t) = \sum_{j=0}^n \mathbf{c}_j t^j, \quad t \in [0, 1]$, minimizzi la seguente funzione obiettivo,

$$\sum_{i=1}^m \|\mathbf{p}_i - \mathbf{X}(t_i)\|_2^2,$$

dove $t_1 = 0$ e, se $i > 1$, si ha

$$t_i = t_{i-1} + \begin{cases} 1/(m-1) & \text{se itype} = 0 \\ \|\mathbf{p}_i - \mathbf{p}_{i-1}\|_2/L & \text{se itype} = 1 \end{cases},$$

con $L = \sum_{i=2}^m \|\mathbf{p}_i - \mathbf{p}_{i-1}\|_2$ (si individuino prima le ascisse e poi le ordinate dei punti $\mathbf{c}_j, j = 0, \dots, n$ risolvendo due analoghi problemi di approssimazione polinomiale nel senso dei minimi quadrati utilizzando le equazioni normali).

- realizzare uno **script** che, posto $n = 7$, definisce i punti nel piano $\mathbf{p}_i = (\cos(\pi t_i), \sin(\pi t_i))/(t_i + 1), i = 1, \dots, 30$ con i $t_i, i = 1, \dots, 30$ parametri equispaziati fra 0 e 4. Quindi, richiamando la **function** costruita al punto precedente prima con **itype** = 0 e poi con **itype** = 1, fare in modo che tale **script** disegni in due diverse figure le due curve parametriche polinomiali $\mathbf{X}(t)$ di grado 7 ottenute (usare la stessa scala sui due assi). Commentare i risultati.

Esercizio 3

- Scrivere una **function** Matlab che implementa il metodo iterativo SOR (variante di Gauss-Siedel vista a lezione che utilizza un parametro ω di rilassamento) per sistemi lineari (assegnare in input una tolleranza **tol** per il criterio di arresto e il numero massimo **nmax** di iterazioni da eseguire). Utilizzare il residuo relativo per definire il criterio di arresto.
- Realizzare uno **script** che definisce nel formato compresso di Matlab per le matrici sparse una matrice A di dimensione $n \times n$ con $n = 1000$, avente una densità di elementi non nulli random fra 0 e 1 di circa $1/n$ e in cui $A(i, i)$ sia pari a 2 più la somma degli altri elementi non nulli sulla riga $i = 1, \dots, n$ (si vedano i comandi `sprand` e `spdiags`). Fare in modo che lo **script** approssimi la soluzione di $A\mathbf{x} = \mathbf{b}$, con $b_i = 1 + \cos(2\pi i/n), i = 1, \dots, n$, ripetendo per 10 volte l'esperimento (al variare della generazione random della matrice A) e, ogni volta, scriva in un file di report l'errore relativo (in norma euclidea) effettivamente raggiunto (comando Matlab “\”

per calcolare la soluzione "esatta") e il numero di iterazioni eseguite da SOR (usare il vettore nullo come innesco) per i tre valori di $\omega = 0.5, 1, 1.1$ (porre $nmax = 500$ e $tol = 1e - 08$). Fare anche in modo che al termine lo **script**, per ogni valore di ω considerato, aggiunga nel file di report le medie aritmetiche (fatte sulle 10 diverse definizioni random della matrice) degli errori ottenuti e del numero di iterazioni eseguite e che infine stampi una figura che visualizzi il pattern dell'ultima matrice A costruita con scritto sotto il suo numero di condizionamento in norma 2 (usare il comando **cond** applicato a **full(A)**). Commentare i risultati (legame fra residuo relativo ed errore relativo, utilità dell'utilizzo del parametro ω , variazione del numero di iterazioni eseguite e dell'errore rispetto alle 10 diverse definizioni random della matrice).

Esercizio 4 Scrivere una **function** Matlab di nome **newtons** per risolvere un sistema nonlineare del tipo

$$F(\mathbf{x}) = 0, \quad F: \mathbb{R}^n \rightarrow \mathbb{R}^n,$$

mediante il metodo di Newton e quello delle corde (detto anche Newton stazionario), con le seguenti specifiche.

- Dati in ingresso: vettore iniziale $\mathbf{x}$, variabili **F** ed **F1** relative alle *function handles* per la funzione F e la sua Jacobiana, massimo numero d'iterazioni **iter_max** consentito, matrice riga **tol** = $[\tau_a \ \tau_r]$ di due tolleranze (assoluta e relativa), variabile **m** (tale che: se **m** = 0 la funzione implementa il metodo delle corde, altrimenti implementa quello di Newton).
- Risultati in uscita: vettore approssimazione finale $\mathbf{x}$, numero d'iterazioni effettuate **iter**, vettore **normF_rec** con le norme infinito dei valori di F nelle varie iterate (compreso quello al punto d'innescio), il vettore **condF1_rec** con i valori dei numeri di condizionamento della Jacobiana sulle varie iterate (compreso quello al punto d'innescio).
- Implementazione: oltre alla salvaguardia data da **iter_max**, considerare il seguente criterio di arresto basato sul residuo:

$$\|F(\mathbf{x}^{(k)})\|_{\infty} \leq \tau_a + \tau_r \|F(\mathbf{x}^{(0)})\|_{\infty};$$

inoltre, risolvere il sistema lineare generato ad ogni iterazione del metodo mediante opportuno uso del comando Matlab "****"; prevedere un messaggio finale che renda conto dei motivi di arresto della funzione rispetto ai criteri adottati.

Si consideri, quindi, il seguente sistema nonlineare:

$$F(\mathbf{x}) =: F((x_1, x_2, x_3, x_4)^{\top}) =: \begin{pmatrix} 10(x_1^2 - x_2^2) + x_1 - 0.5 \\ 10(x_2^2 - x_3^2) + x_2 - x_1 \\ 10(x_3^2 - x_4^2) + x_3 - x_2 \\ 5(x_4^2 - 0.04) + x_4 - 0.3 \end{pmatrix} = \mathbf{0} \in \mathbb{R}^4,$$

originato nell'ambito dell'ingegneria chimica da un tipico sistema di 4 reattori chimici CSTR (*Continuous Stirred-Tank Reactor*) in serie, con diffusione nonlineare. Il sistema ha per incognite le concentrazioni x_i (adimensionali) di una data sostanza in uscita dal reattore $i \in \{1, \dots, 4\}$.

Definire uno *script* Matlab che richiami opportunamente la **newtons** sui punti d'innescio

$$\mathbf{x}_0 = (0, 0, 0, 0)^{\top}, \quad \mathbf{x}_1 = (1, 1, 1, 1)^{\top},$$

sia con **m** = 0 che con **m** ≠ 0, con una salvaguardia di 66 iterazioni e $\tau_a = \tau_r = 10^{-9}$, visualizzando il numero d'iterazioni effettuate e l'errore relativo $\frac{\|\mathbf{x} - \mathbf{x}^*\|_{\infty}}{\|\mathbf{x}^*\|_{\infty}}$ a terminazione ottenuto in ciascun caso.

Per il calcolo dell'errore relativo, utilizzare come valore di $\mathbf{x}^*$ quello restituito dalla funzione predefinita Matlab **fsolve** intorno a $(1, 1, 1, 1)^{\top}$. Per ciascuno dei due inneschi, deve essere generata una distinta figura rappresentante l'andamento della norma infinito di F nelle varie iterate in funzione del numero di iterazioni effettuate (a partire dal punto d'innescio, considerando la scala logaritmica sull'asse delle ordinate) e, infine, una terza figura deve riportare l'andamento del numero di condizionamento della Jacobiana (in norma infinito e a partire dal punto d'innescio) nelle varie iterazioni del metodo di Newton, quando applicato a partire dal punto d'innescio $(1, 1, 1, 1)^{\top}$. Nelle tre figure, inserire titolo, etichette sugli assi, griglia di base e legenda. Interpretare i risultati ottenuti.